

Practical 1: Basic Input/Output & Conditionals

1(A) Find the year in which a person will turn 100 years old.
Program

```
from datetime import datetime

# Get current year
current_year = datetime.now().year

# Input user name and age
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# Calculate 100th year
year_100 = current_year + (100 - age)

print(name + ", you will turn 100 years old in the year", year_100)
```

1(B) Check whether a number is even or odd.
Program

```
num = int(input("Enter a Number: "))

if num % 2 == 0:
    print("The number is even.")
else:
    print("The number is odd.")
```

1(C) Determine grade based on SGPI score.
Program

```
sgpi = float(input("Enter your SGPI: "))

if 9.00 <= sgpi <= 10.00:
    print("Grade: O")
elif 8.00 <= sgpi <= 8.99:
    print("Grade: A+")
elif 7.00 <= sgpi <= 7.99:
    print("Grade: A")
elif 6.00 <= sgpi <= 6.99:
    print("Grade: B+")
elif 5.50 <= sgpi <= 5.99:
    print("Grade: B")
elif 5.00 <= sgpi <= 5.49:
    print("Grade: C")
elif 4.00 <= sgpi <= 4.99:
    print("Grade: P")
elif sgpi < 4.00:
    print("Grade: F")
else:
    print("Invalid SGPI! Please enter value between 0 and 10.")
```

Practical 2: Control Structures & Loops

2(A)

Qn. Generate the Fibonacci series up to N terms.

Program

```
n = int(input("Enter the Number of terms: "))
a = 0
b = 1

print("Fibonacci series:")
for i in range(n):
    print(a, end=" ")
    c = a + b
    a = b
    b = c
print()
```

2(B)

Qn. Find the sum of digits of a given number.

Program

```
num = int(input("Enter a number: "))
sum_digits = 0
temp = num

while temp > 0:
    digit = temp % 10
    sum_digits = sum_digits + digit
    temp = temp // 10

print("Sum of digits:", sum_digits)
```

Practical 3: Native Array & List Operations

3(A) Perform basic arithmetic, indexing, and slicing on arrays.
Program

```
arr1 = [10, 20, 30, 40, 50]
arr2 = [1, 2, 3, 4, 5]

print("Array 1:", arr1)
print("Array 2:", arr2)

addition = []
subtraction = []
multiplication = []
division = []

for i in range(len(arr1)):
    addition.append(arr1[i] + arr2[i])
    subtraction.append(arr1[i] - arr2[i])
    multiplication.append(arr1[i] * arr2[i])
    division.append(arr1[i] / arr2[i])

print("Addition:", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)

print("\nSlicing:")
print("Elements from index 1 to 3:", arr1[1:4])
print("Elements from beginning to index 2:", arr1[:3])
print("Elements from index 2 to End:", arr1[2:])
print("Every second element:", arr1[::2])
print("Elements in reverse order:", arr1[::-1])
```

3(B) Implement mathematical functions on arrays.
Program

```
arr = [10, 20, 30, 40, 50]

print("Array:", arr)
print("Sum      =", sum(arr))
print("Maximum  =", max(arr))
print("Minimum  =", min(arr))
print("Number of Elements:", len(arr))

average = sum(arr) / len(arr)
print("Average =", average)

square = [i ** 2 for i in arr]
cube = [i ** 3 for i in arr]

print("Square of elements =", square)
print("Cube of elements =", cube)
```

3(C) Perform array aliasing and copying operations.
Program

```
arr1 = [10, 20, 30, 40, 50]
arr2 = arr1          # Aliasing (Reference Copy)
arr3 = arr1.copy()   # Shallow Copy

arr1[0] = 100

print("Original Array:", arr1)
print("Aliased Array:", arr2)
print("Copied Array:", arr3)
```

Practical 4: NumPy Library Operations

4(A) **Program**
Aim: Basic, advanced, and boolean indexing on NumPy arrays.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
print("Original Array:", arr)

# Basic Indexing & Slicing
print("First Element:", arr[0])
print("Fourth Element:", arr[3])
print("Last Element:", arr[-1])
print("Element from index 2 to 5:", arr[2:6])
print("Reverse Array:", arr[::-1])

# Advanced & Boolean Indexing
print("Elements at indices [1, 3, 5]:", arr[[1, 3, 5]])
print("Elements greater than 40:", arr[arr > 40])
print("Even Elements:", arr[arr % 20 == 0])
```

4(B) **Program**
Aim: Analyze dimensions and structural attributes of arrays.

```
import numpy as np

arr = np.array([[10, 20, 30], [40, 50, 60]])
print("Array:\n", arr)

print("Dimensions (ndim):", arr.ndim)
print("Shape:", arr.shape)
print("Size (total elements):", arr.size)
print("Data type (dtype):", arr.dtype)
print("Item Size (bytes):", arr.itemsize)
print("Total memory used (bytes):", arr.nbytes)
```

Practical 5: Functions & Recursion

5(A) Qn. Check if a given number is Palindrome and Armstrong.
Program

```
def is_palindrome(num):
    temp = num
    rev = 0
    while temp > 0:
        digit = temp % 10
        rev = rev * 10 + digit
        temp //= 10

    if rev == num:
        print(num, "is a Palindrome number.")
    else:
        print(num, "is not a Palindrome number.")

def is_armstrong(num):
    temp = num
    digits = len(str(num))
    total = 0

    while temp > 0:
        digit = temp % 10
        total += digit ** digits
        temp //= 10

    if total == num:
        print(num, "is an Armstrong number.")
    else:
        print(num, "is not an Armstrong number.")

# Main Program
num = int(input("Enter a number: "))
is_palindrome(num)
is_armstrong(num)
```

5(B) Qn. Find the factorial of a number using recursion.
Program

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

num = int(input("Enter a Number: "))

if num < 0:
    print("Factorial does not exist for negative numbers.")
else:
    print("Factorial of", num, "is", factorial(num))
```

5(C) Qn. Lambda function to check string prefix character.
Program

```
starts_with = lambda string, ch: string.startswith(ch)

string = input("Enter a String: ")
character = input("Enter the starting character: ")

if starts_with(string, character):
    print("The string starts with", character)
else:
    print("The string character does not start with", character)
```

Practical 6: Modules & Geometry Functions

6(A) Functions to calculate area & 3D volume of Cone / Pyramid.
Program

```
import math

def square_area(radius):
    return radius * radius

def circle_area(radius):
    return math.pi * radius * radius

def pointy_shape_volume(x, y, square_base):
    if square_base:
        return (1/3) * square_area(x) * y
    else:
        return (1/3) * circle_area(x) * y

side = float(input("Enter side of square: "))
height = float(input("Enter height of pyramid: "))
print("Volume of square pyramid:",
      pointy_shape_volume(side, height, True))

radius = float(input("\nEnter radius of circle: "))
height_cone = float(input("Enter height of cone: "))
print("Volume of right circular cone:",
      pointy_shape_volume(radius, height_cone, False))
```

6(B) Count total words and characters in an input string.
Program

```
text = input("Enter a String: ")

characters = len(text)
words = len(text.split())

print("Number of Characters:", characters)
print("Number of words:", words)
```

Practical 7: Data Structures (Lists & Dictionaries)

7(A)

Qn: Check if two lists have at least one common element.

Program

```
def common_member(list1, list2):
    for item in list1:
        if item in list2:
            return True
    return False

list1 = eval(input("Enter elements of First list: "))
list2 = eval(input("Enter elements of second list: "))

result = common_member(list1, list2)
print("Common element found?:", result)
```

7(B)

Qn: Sort a dictionary by its values (Ascending/Descending).

Program

```
my_dict = {
    "apple": 50,
    "banana": 20,
    "orange": 40,
    "grapes": 10
}

ascending = dict(
    sorted(my_dict.items(), key=lambda item: item[1])
)

descending = dict(
    sorted(my_dict.items(), key=lambda item: item[1], reverse=True)
)

print("Original Dictionary:", my_dict)
print("Ascending Order:", ascending)
print("Descending Order:", descending)
```

Practical 8: Tuples & File Handling

8(A) **Program** Qm: Calculate circle area & circumference returning a tuple.

```
import math

def circle(radius):
    area = math.pi * radius * radius
    circumference = 2 * math.pi * radius
    return (area, circumference)

r = float(input("Enter radius: "))
result = circle(r)

print("Tuple returned:", result)
print("Area =", result[0])
print("Circumference =", result[1])
```

8(B) **Program** Qm: Read, write, append, and check text file properties.

```
# Write operation
text_file = open("sample.txt", "w")
text_file.write("Hello, welcome to Python.\nThis is a text file.")
text_file.close()

# Append operation
text_file = open("sample.txt", "a")
text_file.write("\nThis line is appended.")
text_file.close()

# Read & inspect parameters
f = open("sample.txt", "r")
print("Filename:", f.name)
print("Is Closed?", f.closed)
f.close()
print("Is Closed?:", f.closed)
```

8(C) **Program** Qm: Read and display the last N lines of a text file.

```
file = open("sample.txt", "r")
lines = file.readlines()
file.close()

n = int(input("Enter the Number of last lines to read: "))
print("Last", n, "lines of the file:")

for line in lines[-n:]:
    print(line, end="")
```

Practical 9: Regular Expressions & Text Search

9(A)
Q1(A). Search for a word in a file and count occurrences.
Program

```
import re

file = open("sample.txt", "r")
text = file.read()
file.close()

word = input("Enter the word to search: ")

count = len(
    re.findall(
        r'\b' + re.escape(word) + r'\b',
        text,
        re.IGNORECASE
    )
)

print("Occurrences of", word, "=", count)
```

9(B)
Q1(B). Extract hyperlinks from an HTML file using Regex.
Program

```
import re

file = open("sample_html.html", "r")
html = file.read()
file.close()

links = re.findall(r'<a\s+href="(.*?)">', html)

print("Hyperlinks found:")
for link in links:
    print(link)
```

Practical 10: Date, Time & Calendar Operations

10(A)

Aim: Compare two user-provided dates.
Program

```
from datetime import datetime

date1 = input("Enter first date (DD/MM/YYYY): ")
date2 = input("Enter second date (DD/MM/YYYY): ")

d1 = datetime.strptime(date1, "%d/%m/%Y")
d2 = datetime.strptime(date2, "%d/%m/%Y")

if d1 < d2:
    print(date1, "is earlier.")
elif d2 < d1:
    print(date2, "is earlier.")
else:
    print("Both dates are the same.")
```

10(B)

Aim: Calculate execution time of a Python program loop.
Program

```
import time

start = time.time()

total_sum = 0
for i in range(1, 1000001):
    total_sum += i

print("Sum =", total_sum)

end = time.time()
print("Execution time =", end - start, "seconds")
```

10(C)

Aim: Find the first day of a given month and year.
Program

```
import calendar

month = int(input("Enter month (1-12): "))
year = int(input("Enter year: "))

day, num_days = calendar.monthrange(year, month)

weekdays = [
    "Monday", "Tuesday", "Wednesday",
    "Thursday", "Friday", "Saturday", "Sunday"
]

print("The first day of the month is:", weekdays[day])
```

Practical 1: Linear Search Implementation

Searching: $O(N)$

Linearly iterates through an array from left to right until the specified key item is located or the list ends.

Executable Python Implementation

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i # Return index if key found
    return -1 # Key not present

# Execution & Test
data = [12, 45, 67, 23, 89, 34]
key = 23
idx = linear_search(data, key)

print(f"Array: {data}")
print(f"Element {key} found at index: {idx}" if idx != -1 else "Not Found")
```

Practical 2: Stack Data Structure Operations

LIFO Structure

Demonstrates core Stack operations: Push, Pop, Peek, and IsEmpty.

Executable Python Implementation

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        return "Underflow! Stack is empty."

    def peek(self):
        return self.items[-1] if not self.is_empty() else "Stack Empty"

    def is_empty(self):
        return len(self.items) == 0

# Test Stack
st = Stack()
st.push(10)
st.push(20)
st.push(30)

print("Top element:", st.peek())      #Output: 30
print("Popped:", st.pop())            #Output: 30
print("Stack state:", st.items)       #Output: [10, 20]
```

Practical 3: Infix to Postfix Expression Conversion

Stack Application

Converts standard arithmetic infix notation to postfix using operator precedence and a stack.

Executable Python Implementation

```
def infix_to_postfix(expression):
    prec = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}
    stack, output = [], []

    for char in expression:
        if char.isalnum():
            output.append(char)

        elif char == '(':
            stack.append(char)

        elif char == ')':
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            stack.pop()

        else:
            while (
                stack
                and stack[-1] != '('
                and prec.get(stack[-1], 0) >= prec.get(char, 0)
            ):
                output.append(stack.pop())
            stack.append(char)

    while stack:
        output.append(stack.pop())

    return "".join(output)

# Test Conversion
expr = "A+(B*C-(D/E^F)*G)*H"
print("Infix: ", expr)
print("Postfix:", infix_to_postfix(expr))
```

Practical 4: Queue Data Structure Operations

FIFO Structure

Implements Queue operations: Enqueue, Dequeue, Front, and Empty verification.

Executable Python Implementation

```
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        return "Underflow! Queue is empty."

    def front(self):
        return self.queue[0] if not self.is_empty() else "Queue Empty"

    def is_empty(self):
        return len(self.queue) == 0

# Test Queue
q = Queue()
q.enqueue("Job 1")
q.enqueue("Job 2")
q.enqueue("Job 3")

print("Front Job:", q.front())
print("Served:", q.dequeue())
print("Queue state:", q.queue)
```

Practical 5: Bubble Sort Algorithm

Sorting: $O(N^2)$

Repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Executable Python Implementation

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr

print("Bubble Sorted:", bubble_sort([64, 34, 25, 12, 22, 11, 90]))
```

Practical 6: Insertion Sort Algorithm

Sorting: $O(N^2)$

Builds the sorted array one item at a time by shifting larger elements and placing the key in its correct position.

Executable Python Implementation

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr

print("Insertion Sorted:", insertion_sort([12, 11, 13, 5, 6]))
```

Practical 7: Selection Sort Algorithm

Sorting • $O(N^2)$

Divides the list into sorted and unsorted regions and repeatedly selects the minimum element from the unsorted region.

Executable Python Implementation

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_idx = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

print("Selection Sorted:", selection_sort([64, 25, 12, 22, 11]))
```